

Triangular covers of a digital object

Arindam Biswas¹  · Partha Bhowmick² ·
Bhargab B Bhattacharya³ · Barnali Das² ·
Mousumi Dutt⁴ · Apurba Sarkar⁵

Received: 22 September 2017 / Published online: 21 December 2017
© Korean Society for Computational and Applied Mathematics 2017

Abstract An optimal geometric representation of a digital object in the form of minimum-area cover is important in image analysis and computer vision. The first algorithm, to the best of our knowledge, for constructing the minimum-area polygonal cover of a 2D digital object as perceived on a uniform triangular grid, is proposed

A preliminary version of this paper appeared in the Proceedings of IWCIA 2014 [9].

✉ Arindam Biswas
barindam@gmail.com

Partha Bhowmick
bhowmick@gmail.com

Bhargab B Bhattacharya
bhargab@isical.ac.in

Barnali Das
bdbarnalidas@gmail.com

Mousumi Dutt
duttmousumi@gmail.com

Apurba Sarkar
as.besu@gmail.com

- ¹ Department of Information Technology, Indian Institute of Engineering Science and Technology, Shibpur, Howrah, India
- ² Department of Computer Science and Engineering, Indian Institute of Technology, Kharagpur, Kharagpur, India
- ³ Advanced Computing and Microelectronics Unit, Indian Statistical Institute, Kolkata, India
- ⁴ Department of Computer Science and Engineering, St. Thomas College of Engineering and Technology, Kolkata, India
- ⁵ Department of Computer Science and Technology, Indian Institute of Engineering Science and Technology, Shibpur, Howrah, India

here. The polygonal cover is triangular in the sense that its boundary consists of a sequence of edges on the underlying grid. The proposed algorithm is based on certain combinatorial properties of a digital object on a grid, and it computes the tightest cover in time linear in the perimeter of the object. Experimental results are presented to demonstrate the efficacy, robustness, and versatility of the algorithm, and they indicate that the runtime varies inversely with the grid size. The proposed study on triangular covers of digital objects will find many applications to pattern analysis and shape matching.

Keywords Triangular grid · Triangular polygon · Triangular cover · Polygonal cover · Shape analysis

Mathematics Subject Classification 52Cxx

1 Introduction

Optimal polygonal covers of digital objects find diverse applications in many fields of image analysis and computer vision. Such covers are often obtained with a certain approximation parameter or with respect to an underlying grid that determines their precision. The grid is usually defined or characterised by the type of its constituent cells, which may be axis-parallel squares [3], equilateral triangles [8], or isosceles right-angled triangles [30]. Whether polygonal covers can efficiently be generated in low-order polynomial time for different grid patterns is a challenging problem, and this paper is focused on the specific problem when the cells comprising the underlying grid are equilateral triangles.

A *triangular grid*, also called an *isometric grid* [11], is a grid formed by a regular tiling of the plane with equilateral triangles. Given a triangular grid, the *triangular cover* of a digital object placed on the grid corresponds to a collection of equilateral triangles. Such a cover captures the structural and topological information of the concerned object, and finds use in applications in many diverse fields, such as digital image analysis [24], geometric modeling [22], clustering and pattern analysis [3, 22], ecological simulation [2], etc.

Triangular grids and hexagonal grids are duals of each other [24]. There exists a multitude of works in digital geometry mainly in the framework of square grids, as compared to triangular and hexagonal grids. Based on algorithmic comparison, it has been shown in [33] that there is a strong degree of similarity between square grids and hexagonal grids. This is also evidenced from the literature today, as we see several interesting works on characterization and construction of geometric primitives on triangular and hexagonal grids. Some of these, related to digital lines, digital circles, and polygons, may be found in [10, 14, 23, 26, 28].

Some results on tiling in triangular grid have also been reported in recent time. There is no polynomial-time algorithm to solve the finite tiling problem. So, an attempt has been made in [31] to solve the problem using group theory. Such group-theoretic approaches for tiling a given finite region using polyominoes from triangular, square, or hexagonal lattices can be traced back to 1990s [7]. In [12], regular production

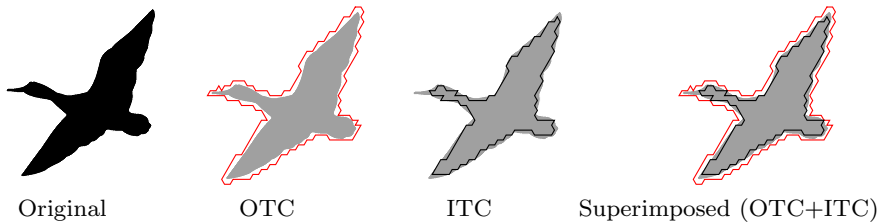


Fig. 1 The outer cover (OTC) and the inner cover (ITC), corresponding to the image Crane

systems are used as a tool for analyzing triangle tilings. In [1], a solution has been provided for the problem of N -tiling of a triangle ABC by a triangle T (tile) such that ABC can be represented as the union of N triangles congruent to T , overlapping only at their boundaries. In [20], a technique has been proposed for regular and irregular tiling of convex polygons with congruent triangles. Another technique proposed in [5] shows how to compute the locations of the vertices of a polygon, so that it can be perfectly tiled by lattice triangles. Tiling has also been implemented with other techniques such as using trichromatic colored-edged triangles, Golden Triangles, and Penrose Rhombs [4,6].

We propose here the first algorithm, to the best of our knowledge, for construction of triangular covers of a digital object for different grid sizes, which is based on certain combinatorial rules. Figure 1 shows triangular covers produced by our algorithm on a typical digital object for two different grid sizes. Although there exist similar works in the literature, no definite algorithm is available till date for the aforesaid problem. For example, the work in [30] is on tiling an image by isosceles right-angled triangles of the same size. Some investigations were made on analyzing the cellular topology with triangular grids, e.g., [25], where coordinate triplets are used to address the triangle pixels of both orientations, the edges between them, and the points at the triangle corners. The main advantage of a topological description is that it provides more information compared to the usual description, storing not only the pixels of the two-dimensional image but also the lower dimensional segments, the edges, and the points separating the pixels. To represent the points of a triangular grid and their neighborhood relations, coordinate triplets can be used, as shown in [24]. For computing isoperimetrically optimal polygons in the triangular grid with Jordan-type neighborhood on the boundary, a method has been proposed in [26].

For construction of optimum-area outer and inner covers of a digital object against a given isothetic or a square grid, an algorithm is proposed in [3]. The algorithm proposed here for the triangular grid is essentially different from the one in [3] mainly for three reasons. First, the combinatorial cases on the triangular grid are different from those on the square grid. Second, the grid lines are non-rectilinear in a triangular grid, thus posing a higher level of difficulty compared to the square grid. Third, all the computations in the square grid can be restricted to integer domain only, whereas, the computations are in real domain in the case of triangular grid. The outer triangular cover and inner triangular cover can be represented by chain codes. The shape analysis can be done using outer triangular covers. The Levenshtein distance [21] between the two chain codes may be used here for shape retrieval. Also, the triangular covers can be used for shape

matching using iso-taxicab distance [16]. The trigonometric function of iso-taxicab distance is described in [16]. The taxicab geometry was introduced by Krause [19] which is a non-Euclidean geometry. The concept of iso-taxicab geometry was introduced by Sowell [29]. There are quite a few studies on iso-taxicab geometry [17, 18].

The problem addressed in this paper is stated as follows. Given a 2D digital object (Definition 1) A registered on a background triangular grid (Definition 2) which consists of three sets of parallel grid lines, which are inclined at 0° , 60° , and 120° w.r.t. x -axis, the problem is to find the tight (outer and inner) triangular covers of A . The shape of the triangular cover varies w.r.t. the grid size (Definition 2) and the registration of the object with the underlying grid. Hence for smaller grid sizes, our work almost traces the contour of the digital object, whereas it significantly reduces the output complexity (i.e., number of vertices) when the cover is generated for higher grid sizes. Figure 1 shows the original Crane image (left), its outer triangular cover (OTC) and the inner triangular cover (ITC) in the middle, and the superimposed outer and inner triangular covers (right).

2 Definitions and Preliminaries

Definition 1 (*Digital object*) A digital object (henceforth referred as an object) is a finite subset of $\mathbb{Z}^2$ consisting of one or more k ($= 4$ or 8)-connected components [15]. In our work, we consider the object A as a single 8-connected component which may contain holes. A hole of A is a k -connected component of $\mathbb{Z}^2 \setminus A$ which is completely surrounded by A [27].

Definition 2 (*triangular grid*): A triangular grid $\mathbb{T} := (\mathbb{L}_{60}, \mathbb{L}_0, \mathbb{L}_{120})$ consists of three sets of parallel grid lines, which are inclined at 60° , 0° , and 120° (w.l.o.g) w.r.t. x -axis [13]. The grid lines in $\mathbb{L}_{60}, \mathbb{L}_0, \mathbb{L}_{120}$ correspond to three distinct coordinates, namely α, β, γ . Three grid lines, one each from $\mathbb{L}_{60}, \mathbb{L}_0, \mathbb{L}_{120}$, intersect at a point in the Euclidean plane called triangular grid point. The Euclidean distance between two consecutive triangular grid points along a triangular grid line is termed as *grid size*, g . A line segment of length g connecting two consecutive triangular grid points on a triangular grid line is called triangular *grid edge*. The smallest-area triangle formed by three grid edges, one each from $\mathbb{L}_{60}, \mathbb{L}_0$, and $\mathbb{L}_{120}$, is called *unit grid triangle* (UGT).

For a given grid point, p , there are six neighboring $UGTs$, given by $\{T_i : i = 0, 1, \dots, 5\}$ as shown in Fig. 2. A portion of the triangular grid is shown in Fig. 2 along with direction codes. It has six distinct regions called sextants, each of which is well-defined by two rays starting from $(0, 0, 0)$. For example, Sextant 1 is defined by the region $\alpha_+ \cap \beta_+$, Sextant 2 is defined by the region $\alpha_- \cap \gamma_-$, and so on.

Definition 3 (*Grid segment*) A line segment of length g connecting two consecutive grid points on a grid line is called *grid edge*.

Definition 4 (*Triangular polygon*) A (finite) polygon Q imposed on the grid $\mathbb{T}$ is termed as a *triangular polygon* if its sides are collinear with lines in $\mathbb{L}_0, \mathbb{L}_{60}$, and $\mathbb{L}_{120}$. It consists of a set of $UGTs$, and is represented by the (ordered) sequence of its vertices, which are grid points. It can also be represented by a sequence of directions

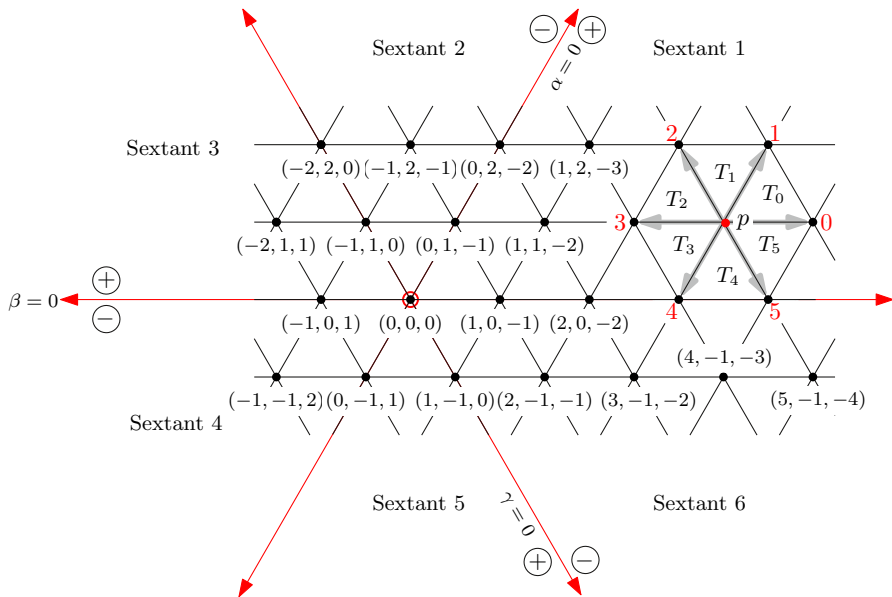


Fig. 2 Portion of a triangular grid, the UGTs $\{T_0, T_1, \dots, T_5\}$ incident at a grid point p , and the direction codes $\{0, 1, \dots, 5\}$ of neighboring grid points of p

and a start vertex. Its *interior* is defined as the set of points with integer coordinates lying inside it. An *outer triangular polygon* is the minimum-area triangular polygon that tightly covers the digital object A in $\mathbb{T}$ i.e., the number of its constituting UGTs is minimum. Each UGT having at least one of its sides on the boundary of the outer polygon has object occupancy. An *inner triangular polygon* is contained in A such that its border is a subset of A and the number of its constituting UGTs is maximum. An *outer triangular hole polygon* is the maximum-area triangular polygon that inscribes a hole or a concavity of A ; that is, A lies outside the outer triangular hole polygon and its border is a subset of $\bar{A}$. An *inner triangular hole polygon* contains a hole and its border is a subset of A .

Definition 5 (*Outer triangular cover*) The outer triangular cover (OTC), $\bar{Q}$, is the set of outer triangular polygons and outer triangular hole polygons, such that the region given by the union of the outer triangular polygons minus the union of the interiors of the outer triangular hole polygons, contains a UGT if and only if it has object occupancy.

Definition 6 (*Inner triangular cover*) The inner triangular cover (ITC), Q , is the set of inner polygons and inner hole polygons, such that the region given by the union of the inner polygons minus the union of the interiors of the inner hole polygons, contains a UGT if and only if it is a subset of A .

Hence, the OTC of an object consists of minimum number of UGTs and its ITC consists of maximum number of UGTs. As a consequence, given an object A and a grid imposed on A , the OTC and ITC are, respectively, of minimum and of maximum areas measured in the grid unit.

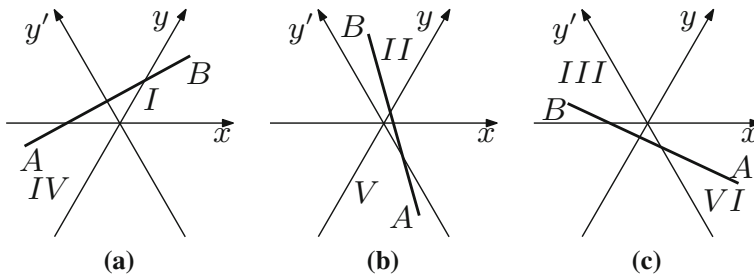


Fig. 3 Two points have (a) I–IV orientation, (b) II–V orientation, and (c) III–VI orientation to one another in iso-taxicab geometry

The proposed algorithm is designed in such a way that it takes into account the local combinatorial arrangements and traverses accordingly without backtracking to produce the tightest cover. Also, the resolution of the background grid can be varied to enable fine or coarse analysis of the underlying shape. The efficiency of the algorithm rests on the fact that it is completely devoid of any backtracking.

An application of OTC for shape analysis is also described. The shape retrieval is performed using chain codes whereas the shape matching is done using iso-taxicab distances [16]. There are three axes in iso-taxicab geometry which are 60° apart, i.e., the three axes coincides with the axes of triangular grid (see Fig. 2). Thus, iso-taxicab distance can be used in triangular grid. In iso-taxicab geometry, an Euclidean unit-circle becomes a unit-radius hexagon. The iso-taxicab geometry has similarity with the contour-segments that we are using in triangular cover. The three orientations of lines in iso-taxicab geometry is shown in Fig. 3. The corresponding distances are as follows.

$$d_I(A, B) = \begin{cases} |x_1 - x_2| + |y_1 - y_2|, & \text{I–IV orientation} \\ |y_1 - y_2|, & \text{II–V orientation} \\ |x_1 - x_2|, & \text{III–VI orientation} \end{cases} \quad (1)$$

The rest of the paper is organized as follows. The review of earlier works and the necessary definitions have already been given in Sects. 1 and 2 respectively. The algorithm for constructing the outer triangular cover and the inner triangular cover have been explained in Sects. 3 and 4 respectively. The proof of correctness of the algorithm is given in Sect. 3.5. Section 3.6 provides the estimation of the running time of the algorithm. Section 5 contains the relevant experimental results on several digital objects. We have used OTCs in shape analysis as explained in Sect. 6. Finally in Sect. 7, we conclude the paper with few open problems and possible future directions.

3 Outer triangular cover for a single connected component

During the construction of the triangular cover $\overline{Q}$ of A , our algorithm traverses the boundary of $\overline{Q}$ with the invariant that A always lies left w.r.t. the direction of traversal. The top-left pixel on the boundary of the object, p_0 , is given as input, from which

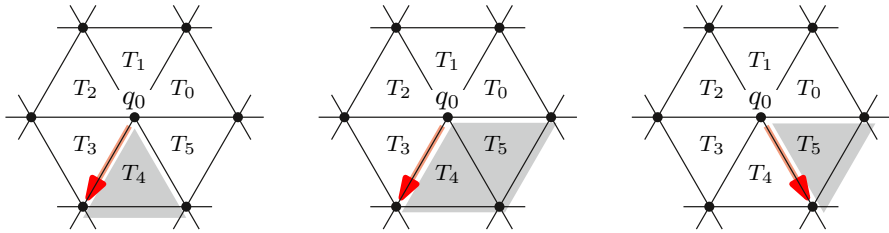


Fig. 4 Determining the start vertex, q_0 , from the top-left object pixel, p_0 , which would lie either in T_4 (left or middle configuration) or in T_5 (middle or right). Object-occupied UGTs are shown in gray, and start directions in red

the start vertex q_0 of $\overline{Q}$ is computed, as shown in Fig. 4. If A contains any hole, then the top-left pixel of the hole boundary is also given as input. However, if the object contains a large concave region with ‘narrow mouth’, then the hole polygon inscribing that concave region is detected and traversed later from a UGT containing the border of the narrow mouth.

3.1 Classification of vertices

The *object occupancy vector* corresponding to $\langle T_i : i = 0, 1, \dots, 5 \rangle$ incident at a grid point q is given by $A_q = \langle a_0 a_1 \dots a_5 \rangle$, where, for $i = 0, 1, \dots, 5$, we have $a_i = 1$ if T_i incident at q has object occupancy, and 0 otherwise. The object occupancy of each T_i is determined by checking its interior integer points lying along its borders. The algorithm then uses the vector A_q to determine whether q lies on $\overline{Q}$ and to compute the edges of $\overline{Q}$ incident at q . For example, $A_q = 000011$ implies T_4 and T_5 are object-occupied. So, the direction of the incoming edge at q becomes 3 and that of the outgoing edge becomes 5, since the object lies left during traversal of $\overline{Q}$.

A grid point q lies on $\overline{Q}$ if A_q contains both 0’s and 1’s. If $A_q = 0^6$, then q lies outside $\overline{Q}$ (Fig. 5a); and $A_q = 1^6$ implies q lies strictly inside $\overline{Q}$ (Fig. 5l). Apart from these two cases, there arise $2^6 - 2 = 62$ combinatorial cases of A_q . These 62 cases can be simplified to following 5 cases only, with their corresponding sub-cases (Fig. 5b–k), depending on the number of 1’s in A_q , which is denoted by $k (= 1, 2, \dots, 5)$.

- $k = 1$: There are 6 sub-cases depending on the position of 1-bit in A_q . For each sub-case, the angle at q is $\pi/3$ (Fig. 5b). If $a_i = 1$, then the *direction pair* representing the incoming and the outgoing edges at q is $((i + 4) \bmod 6, i)$.
- $k = 2$: Let $a_i = a_j = 1$. If $j = (i + 1) \bmod 6$, then there are 6 sub-cases, each with angle at q as $2\pi/3$ and direction pair $((i + 5) \bmod 6, i)$, as shown in Fig. 5c. Otherwise, there are 9 sub-cases, each with q occurring twice in $\overline{Q}$ —once (as a vertex or a non-vertex edge point) on the outer triangular polygon, and once on a triangular hole polygon (Fig. 5d).
- $k = 3$: Following are three possibilities.

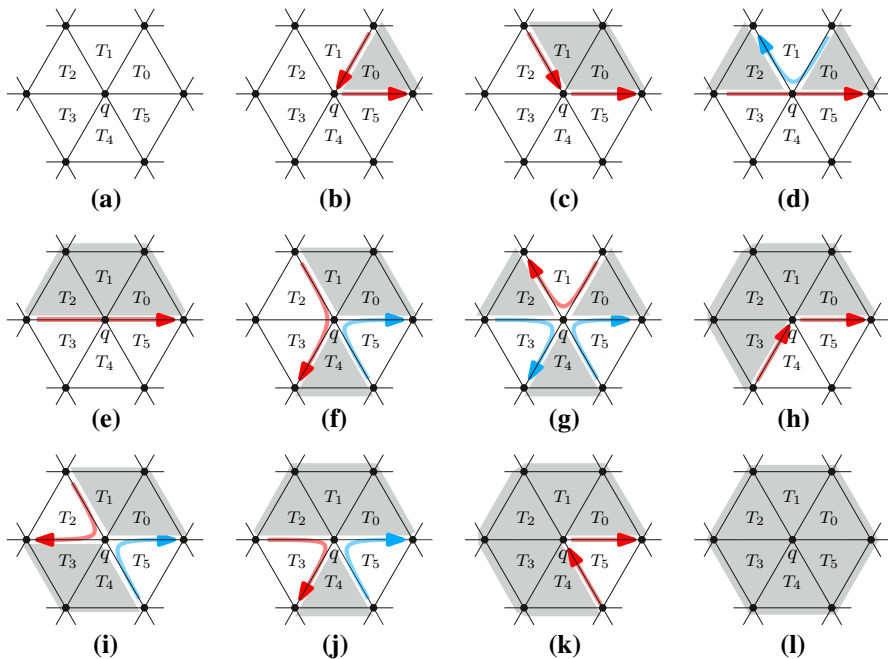


Fig. 5 Classification of a grid point q for **a** $k = 0$, **b** $k = 1$, **c**, **d** $k = 2$, **e–g** $k = 3$, **h–j** $k = 4$, **k** $k = 5$, **l** $k = 6$. A red line indicates the traversal direction of the outer triangular polygon, and a blue line indicates that of an outer triangular hole polygon. Detailed explanations are given in Sect. 3.1

- The positions of three 1-bits in A_q are i , $(i + 1) \bmod 6$, and $(i + 2) \bmod 6$. There are 6 sub-cases in which q is not a vertex but a grid point on an edge of $\overline{Q}$, and with direction pair (i, i) ; see Fig. 5e.
- Two 1-bits have positions i and $(i + 1) \bmod 6$, and the third 1-bit in any position j other than $(i + 2) \bmod 6$. There are 12 sub-cases, each with q occurring twice in $\overline{Q}$ —once as a vertex of the outer triangular polygon, and once as a vertex of a triangular hole polygon (Fig. 5f).
- All three 1-bits in A_q are non-consecutive. There are 2 sub-cases, each with q occurring thrice in $\overline{Q}$ —once as a vertex of the outer triangular polygon, and twice as vertices of two triangular hole polygons (Fig. 5g).
- $k = 4$: There are three possibilities as follows.
 - The positions of four 1-bits in A_q are $(i + j) \bmod 6$, for $j = 0, 1, 2, 3$ (Fig. 5h). There are 6 sub-cases, each with angle at q as $4\pi/3$ and direction pair $((i + 7) \bmod 6, i)$.
 - The positions of two 1-bits are i and $(i + 1) \bmod 6$, and those of the other two 1-bits are j and $(j + 1) \bmod 6$, where $j = (i + 3) \bmod 6$ (Fig. 5i). There are 3 sub-cases, each with q occurring twice in $\overline{Q}$ —once as a vertex of the outer triangular polygon, and once as a vertex of a triangular hole polygon.
 - The positions of three 1-bits are i , $(i + 1) \bmod 6$, and $(i + 2) \bmod 6$, and that of the fourth 1-bit is $(i + 4) \bmod 6$ (Fig. 5j). There are 6 sub-cases, each with q occurring twice in $\overline{Q}$, as in the previous case.

- $k = 5$: There are 6 sub-cases depending on the position of 0-bit in A_q (Fig. 5k). For each sub-case, the angle at q is $5\pi/3$. If $a_i = 0$, then the direction pair representing the incoming and the outgoing edges at q is $((i + 2) \bmod 6, i)$.

3.2 Determining the next direction

The outgoing direction d' at a grid point q lying on $\overline{Q}$ is computed from the incoming direction d and $A_q := \langle a_0 a_1 \dots a_5 \rangle$. It is based on the following observation: The incoming direction d implies $a_j = 1$ and $a_{(j+1) \bmod 6} = 0$, where $j = (d + 2) \bmod 6$. Hence, j is incremented (in modulo 6 arithmetic) until the next 1-bit in A_q is encountered, say at j' , so that $a_{j'} = 1$. The outgoing direction is then given by $d' = j'$. For example, in Fig. 5f, $A_q = 110010$ and $d = 5$ gives $j = (5 + 2) \bmod 6 = 1$, implying $a_1 = 1$ and $a_2 = 0$; since $a_3 = 0$ and $a_4 = 1$, we get $d' = 4$ for the outer triangular polygon. While traversing the hole polygon in Fig. 5f, $d = 2$ gives $j = (2 + 2) \bmod 6 = 4$, implying $a_4 = 1$ and $a_5 = 0$; since $a_0 = 1$, we get $d' = 0$.

Regarding the start point and type of vertex on OTC we have the following observations.

Observation 3.1 The start point of the OTC is either a 60^0 or 120^0 vertex or an edge point.

Observation 3.2 The number of 60^0 , 120^0 , 240^0 and 300^0 vertices of the OTC always satisfy the equation given by

$$(\#120^0 - \#240^0) + 2(\#60^0 - \#300^0) = 6$$

where $\#x^0$ indicates the number of x^0 vertices.

3.3 Tracing the polygon

During the traversal of (the boundary of the polygon in) $\overline{Q}$, the direction d_c from the current grid point q_c , and the coordinates (i_c, j_c) of q_c , being known, the coordinates of the next grid point is computed as, $q_n := (i_n, j_n)$, lying on $\overline{Q}$, as follows:

$$\begin{aligned} d_c = 0: & \quad i_n = i_c + g \\ & \quad j_n = j_c \\ d_c = 1: & \quad i_n = i_c + g/2 \\ & \quad j_n = j_c - \sqrt{3}/2g \\ d_c = 2: & \quad i_n = i_c - g/2 \\ & \quad j_n = j_c - \sqrt{3}/2g \\ d_c = 3: & \quad i_n = i_c - g \\ & \quad j_n = j_c \\ d_c = 4: & \quad i_n = i_c - g/2 \\ & \quad j_n = j_c + \sqrt{3}/2g \\ d_c = 5: & \quad i_n = i_c + g/2 \\ & \quad j_n = j_c + \sqrt{3}/2g \end{aligned}$$

The vertex type t_n of q_n is computed by determining the object occupancy of its neighboring UGTs and storing them in the object occupancy vector. The direction of traversal, d_n , from q_n is given by

$$d_n = (d_c + 6 - t_n + 3) \bmod 6 \quad (2)$$

The coordinates of each grid point lying on $\overline{Q}$ are computed until q_n coincides with q_s . If the grid point is a vertex then it is added to the sequence of the vertices comprising $\overline{Q}$. The following theorems justify the correctness of $\overline{Q}$ (Definition 5) and how the traversal converges to obtain the sequence of its vertices.

3.4 Algorithm to construct $\overline{Q}$

Algorithm 1 (CONSTRUCT- OTC) shows the important steps to construct the outer triangular polygon of a digital object (single connected component without holes). It takes the object, A , the triangular grid, $\mathbb{T}$, and the top-left pixel of $p_0 \in A$, as input, and generates the sequence of vertices of $\overline{Q}$ as output. The same algorithm is also used to construct the other polygons, e.g., outer triangular hole polygons and also other outer triangular polygons in case of a digital object with multiple components. For each component or each hole, the top-left pixel is supplied as an input.

In Step 1 of the algorithm, the top-left grid point (start vertex) of $\overline{Q}$, namely q_0 , is determined from p_0 using the procedure FIND- START. Its related details have already been explained in Sect. 3. The list L stores the sequence of vertices of $\overline{Q}$. It is initialized with q_0 in Step 2. In Step 3, the start direction is determined based on the UGT containing p_0 , as explained earlier and illustrated in Fig. 4. In the **do-while** loop (Steps 4–9), the outgoing direction d' and the next vertex are determined, until the next vertex coincides with the start vertex.

The outgoing direction d' at q is determined by the procedure FIND- DIR in Step 5. Its related operations are explained in Sect. 3.2. From d' , the next vertex is easily computed by NEXT- VERT in Step 6, using $\mathbb{T}$ and the coordinates of q . The ordered list of vertices, L , is returned finally in Step 10.

Algorithm 1: CONSTRUCT- OTC

Input : $A, \mathbb{T}, p_0$
Output: L

```

1  $q \leftarrow q_0 \leftarrow \text{FIND- START}(A, \mathbb{T})$ 
2  $L \leftarrow \{q_0\}$ 
3  $d \leftarrow \text{FIND- START- DIR}(q_0)$ 
4 do
5    $d' \leftarrow \text{FIND- DIR}(d, q)$ 
6    $q \leftarrow \text{NEXT- VERT}(d', q)$ 
7    $d \leftarrow d'$ 
8    $L \leftarrow L \cup \{q\}$ 
9 while  $q \neq q_0$ ;
10 return  $L$ 
```

Algorithm 2: CONSTRUCT- ITC

Input : $A', \mathbb{T}, p_0$
Output: L

```

1  $q \leftarrow q_0 \leftarrow \text{FIND-START}(A', \mathbb{T})$ 
2  $L \leftarrow \{q_0\}$ 
3  $d \leftarrow \text{FIND-START-DIR}(q_0)$ 
4 do
5    $d' \leftarrow \text{FIND-DIR}(d, q)$ 
6    $q \leftarrow \text{NEXT-VERT}(d', q)$ 
7    $d \leftarrow d'$ 
8    $L \leftarrow L \cup \{q\}$ 
9 while  $q \neq q_0$ ;
10 return  $L$ 
```

Procedure Find-Start($A, \mathbb{T}$)

Input : $A, \mathbb{T}$
Output: q_0

```

1  $m \leftarrow 0$ 
2 for each grid point  $q \in \mathbb{T}$  in row-major order
3 do
4   for  $i = 0$  to 5 do
5     if  $T_i^q \cap A \neq \emptyset$  then
6        $m \leftarrow m + 1$ 
7       goto 11
8     else
9        $m \leftarrow 0$ 
10 while  $m = 0$ ;
11 return  $q$ 
```

Procedure Find-Start-Dir(q_0)

Input : q_0
Output: d

```

1  $m \leftarrow 0$ 
2 for  $i = 0$  to 5 do
3   if  $T_i^{q_0} \cap A \neq \emptyset$  then
4      $m \leftarrow m + 1$ 
5 if  $m = 1$  then
6    $d \leftarrow 5$ 
7 else if  $m = 2$  then
8    $d \leftarrow 4$ 
9 else
10   $d \leftarrow 3$ 
11 return  $d$ 
```

Procedure Find-Dir(d, q)

Input : d, q
Output: d'

```

1 int  $occ\_vec[6]$ ,  $m \leftarrow 0$ ,  $j$ 
2 for  $i = 0$  to 5 do
3   if  $T_i^q \cap A \neq \emptyset$  then
4      $occ\_vec[i] \leftarrow 1$ 
5   else
6      $occ\_vec[i] \leftarrow 0$ 
7 for  $i = 0$  to 5 do
8   if  $occ\_vec[i] = 1$  then
9      $m \leftarrow m + 1$ 
10  $j \leftarrow (d + 2) \bmod 6$ 
11 while  $m - 1 > 0$  do
12   if  $j = 0$  then
13      $j \leftarrow 5$ 
14   else
15      $j \leftarrow j - 1$ 
16   if  $occ\_vec[j] = 1$  then
17      $m \leftarrow m - 1$ 
18   else
19      $occ\_vec[j] = 1$ 
20 for  $i = 0$  to 5 do
21   if  $occ\_vec[i] = 1$  then
22      $m \leftarrow m + 1$ 
23  $d' \leftarrow (d + 6 - m + 3) \bmod 6$ 
24 return  $d'$ 

```

3.5 Proof of correctness

To prove that the algorithm CONSTRUCT- OTC (Algorithm 1) indeed generates the outer triangular cover, we have to prove that the cover generated by the algorithm is of minimum area and the algorithm terminates. Aforementioned criterion follows from following two theorems.

Theorem 1 *The outer triangular cover, $\overline{Q}$, is tight i.e., the area of $\overline{Q}$ is minimum.*

Proof From the combinatorial cases explained in Sect. 3.1 and from the strategy of getting outgoing directions explained in Sect. 3.2, we can make the following observation: For every grid edge e lying on $\overline{Q}$, one UGT incident on e has object-occupancy and the other has not. Further, among these two UGT s, the interior of only the former lies inside $\overline{Q}$. This implies that none of the UGT s whose edges lie on $\overline{Q}$ and interiors lie inside $\overline{Q}$, can be removed from $\overline{Q}$, which proves the minimum-area criterion of $\overline{Q}$ produced by Algorithm 1. $\square$

Theorem 2 *The construction of outer triangular cover, $\overline{Q}$, concludes at the start vertex.*

Procedure Next-Vert(d', q)

Input : d', q
Output: q

```

1 if  $d' = 0$  then
2    $i_n \leftarrow i_c + g$ 
3    $j_n \leftarrow j_c$ 
4 if  $d' = 1$  then
5    $i_n \leftarrow i_c + g/2$ 
6    $j_n \leftarrow j_c - \sqrt{3}/2g$ 
7 if  $d' = 2$  then
8    $i_n \leftarrow i_c - g/2$ 
9    $j_n \leftarrow j_c - \sqrt{3}/2g$ 
10 if  $d' = 3$  then
11    $i_n \leftarrow i_c - g$ 
12    $j_n \leftarrow j_c$ 
13 if  $d' = 4$  then
14    $i_n \leftarrow i_c - g/2$ 
15    $j_n \leftarrow j_c + \sqrt{3}/2g$ 
16 if  $d' = 5$  then
17    $i_n \leftarrow i_c + g/2$ 
18    $j_n \leftarrow j_c + \sqrt{3}/2g$ 
    
```

Proof To prove that the start vertex q_0 is finally reached when the algorithm terminates, observe that the algorithm maintains the invariant that the object A always lies left during traversal. Since the incoming direction is left out while commencing the start direction from q_0 , the outgoing direction from the last grid point on $\overline{Q}$ finally leads to q_0 , hence ensuring its successful termination. $\square$

3.6 Time complexity

For time complexity analysis, let n be the number of pixels comprising the contour (border) of a connected component comprising A . A UGT may contain as few as one contour pixel, but for every seven UGT s through which the contour passes in succession, there would be at least $O(g)$ contour pixels, as g is the grid size of $\mathbb{T}$. Hence, there are at most $O(n/g)$ UGT s containing the object contour, or, the number of grid points on $\overline{Q}$ is at most $O(n/g)$.

To find the start vertex q_0 and the corresponding start direction, $O(1)$ time is required. So, Steps 1–3 of Algorithm 1 takes $O(1)$ time. Each UGT is checked in $O(g)$ time, as explained in Sect. 3.1. As each grid point has six neighboring UGT s, the time complexity to compute the object occupancy vector at each grid point q lying on $\overline{Q}$ is $6 \cdot O(g) = O(g)$. Thus, for q , outgoing direction and next grid point finding are performed in $O(g)$ time. Hence, the *worst-case* time complexity of Algorithm 1 is $O(n/g) \cdot O(g) = O(n)$.

For the *best-case* analysis, observe that there can be $O(g^2)$ contour pixels in a UGT , which implies that there can be $O(n/g^2)$ UGT s containing the object contour,

or, the number of grid points on $\overline{Q}$ is $O(n/g^2)$. As each UGT can be checked for object occupancy in $O(g)$ time, we get the time complexity as $O(n/g^2) \cdot O(g) = O(n/g)$.

4 Inner triangular cover

The inner cover, $\underline{Q}$, can be constructed by the method of generating $\overline{Q}$, but in the reverse manner. A grid point $q \in A$ is classified as a vertex of $\underline{Q}$, if and only if at least one of the six UGTs incident at q is fully occupied by the object A i.e., $T_i^q \cap A = T_i^q$ where $i \in \{0, 1, 2, 3, 4, 5\}$.

Algorithm 2 states the algorithm CONSTRUCT-ITC for constructing the inner triangular cover of an object A , with or without holes. The algorithm is similar to CONSTRUCT-OTC, but it takes the background A' as input. The construction of $\underline{Q}$ keeps A' to the left during the traversal. The polygon is traced to the next grid point q_n . The type of q_n is decided and the direction of traversal from q_n is computed. The traversal is continued until the start vertex, q_s , is reached.

5 Experimental results

The proposed algorithm is implemented in C in Ubuntu 12.04, 32-bit, kernel version 3.5.0-43-generic, the processor being Intel core i5-3570, 3.40 GHz FSB. The algorithm has been used to construct triangular covers of digital objects in various binary images of logo datasets, real-world objects, and geometric shapes, for different grid sizes.

The outer triangular covers (OTCs) of three different objects are shown in Fig. 6. The outer triangular polygons are shown in red and outer hole polygons in blue. For `Lincoln` image, there are five outer hole polygons along with the outer triangular polygon. It may be noticed that there are a few small holes in the top-left region of the digital object of `Lincoln` image; however, only one outer hole polygon in that region is reported by our algorithm, as the other holes are not large enough to accommodate any polygon for $g = 20$.

Figure 7 shows the outer triangular covers of five objects from our datasets. The second and the fourth ones are geometric shapes, the third one is a logo image, and the first and the fifth are real-world objects. In each row, the input object, the number of pixels constituting its border (n), the OTCs for some grid sizes, and the number of vertices (n_v) of the OTCs are shown. The results indicate that n_v decreases with the increase of g . As g decreases, smaller irregularities on the object border do not leave much impression on $\overline{Q}$, but the gross shape of the object gets captured through fewer vertices. The cover $\overline{Q}$ captures the structural property of the object for smaller values of g , and it loses information with increasing values of g . For higher values of g , output complexity is low, resulting in less storage, but topological information is not always retained. For example, some of the outer triangular hole polygons may disappear for higher values of grid size. Such a collection of $\overline{Q}$ for changing grid size might be useful for multi-scale shape analysis.

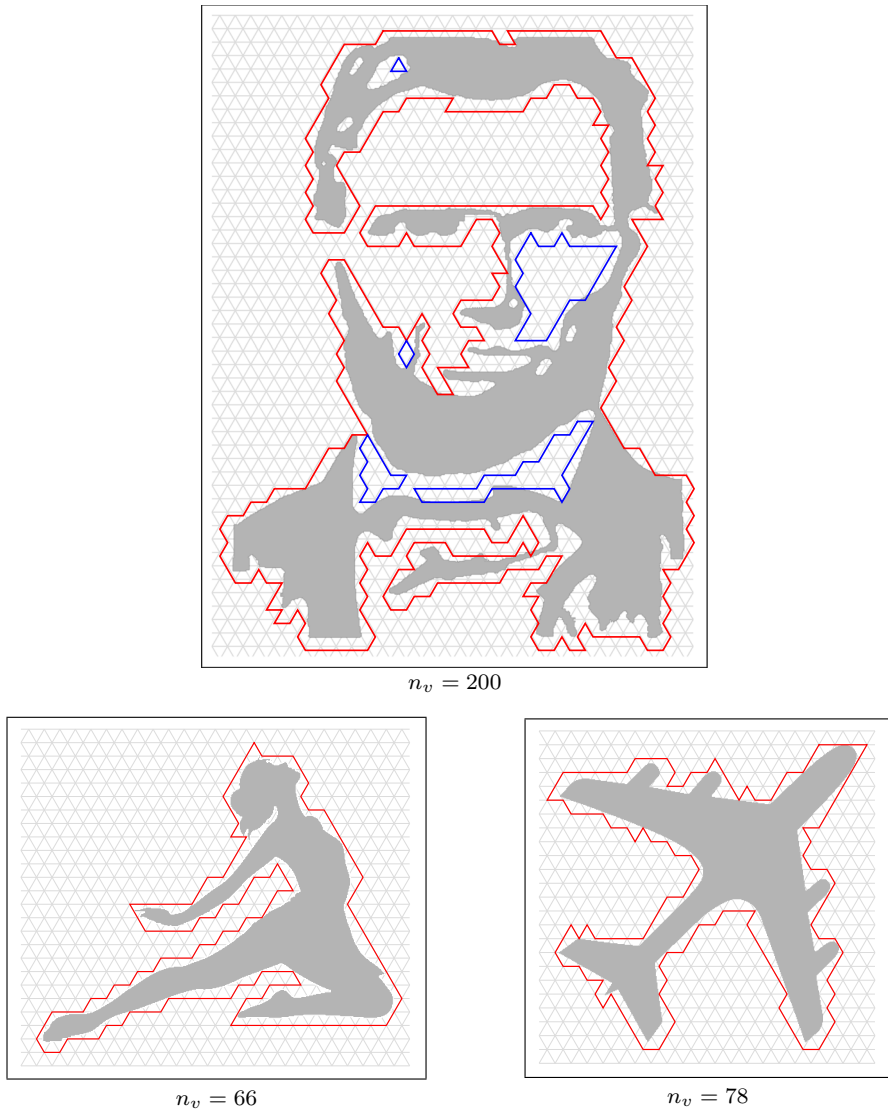


Fig. 6 Triangular covers produced by the proposed algorithm on Lincoln, dancer, and aircraft images, with $g = 20$

Table 1 presents a consolidated information of the OTCs for several objects at different grid sizes. In all cases, the number of vertices decreases with g , which is also evident from the plots showing n_v versus g in Fig. 8. The CPU times plotted against different grid sizes for four images are shown in this figure. As evident from these plots, the CPU time taken to compute the OTCs falls sharply with the increasing value of g , which indicates that the runtime of the algorithm on a real-world image follows the best-case runtime complexity of $O(n/g)$, as explained in Sect. 3.6.

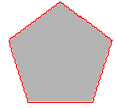
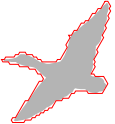
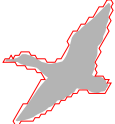
				
Baby $n = 3024$	$g = 1$ $n_v = 2976$	$g = 6$ $n_v = 243$	$g = 12$ $n_v = 129$	$g = 22$ $n_v = 71$
				
Pentagon $n = 1824$	$g = 6$ $n_v = 127$	$g = 12$ $n_v = 66$	$g = 18$ $n_v = 46$	$g = 22$ $n_v = 39$
				
Crane $n = 3178$	$g = 8$ $n_v = 187$	$g = 10$ $n_v = 143$	$g = 14$ $n_v = 97$	$g = 20$ $n_v = 82$
				
Letter_T $n = 2100$	$g = 4$ $n_v = 600$	$g = 10$ $n_v = 108$	$g = 16$ $n_v = 88$	$g = 22$ $n_v = 50$
				
Bear $n = 1240$	$g = 6$ $n_v = 120$	$g = 10$ $n_v = 52$	$g = 14$ $n_v = 32$	$g = 16$ $n_v = 40$

Fig. 7 Experimental results for different grid sizes. (n = number of pixels on the object contour; n_v = number of vertices of $\bar{Q}$.)

To estimate the effectiveness of triangular covers in capturing the object shape, a comparison of the proposed algorithm has been done with the algorithm for finding isothetic covers proposed in [3]. Figure 9 shows this comparison for a geometric shape (Letter_T), a logo image (Kangaroo), and a real-world object (Crane), all for $g = 16$.

The object in Letter_T is perfectly aligned with the horizontal and the vertical lines of the isothetic grid. Hence, its isothetic cover has fewer vertices compared to its triangular cover; however, in area measure, the triangular cover is better compared to the isothetic cover. For the image Kangaroo, the number of vertices of the triangular cover is more than that of the isothetic cover; but the area of the triangular cover is comparatively less, indicating that it is tighter than the isothetic cover, and hence more compact. The boundary of the object in Crane has some portions aligned

Table 1 Summary of experimental results

Object A	$ A $	n	g	n_{60}	n_{120}	n_{240}	n_{300}	n_v	n_T	$ \bar{Q} $	$p_{\bar{Q}}$	T
Baby	132, 629	3024	4	45	159	149	47	400	19, 760	136, 901	2744	600
			8	26	81	69	29	205	5093	141, 141	2760	410
			12	2	68	52	7	129	2350	146, 531	2568	360
			16	4	46	30	9	89	1348	149, 427	2592	340
			20	3	35	21	7	66	887	153, 633	2540	320
Pentagon	116, 711	1636	24	2	31	19	5	57	619	154, 388	2472	310
			4	19	81	77	18	195	17, 185	119, 061	1516	530
			8	10	40	36	9	95	4401	121, 964	1528	390
			12	0	36	30	0	66	2004	124, 957	1464	360
			16	2	26	22	1	51	1152	127, 701	1504	340
Aircraft	50, 607	2724	20	3	22	16	3	44	745	129, 038	1540	330
			24	2	18	12	2	34	537	133, 936	1560	320
			4	44	131	123	45	343	7835	54, 282	2460	520
			8	23	75	57	29	184	2086	57, 809	2496	400
			12	5	61	43	11	120	998	62, 229	2352	350
			16	5	46	38	6	95	593	65, 735	2320	340
			20	8	34	28	8	78	396	68, 589	2360	320
			24	4	30	24	4	62	286	71, 333	2208	320

Table 1 continued

Object A	A	n	g	n ₆₀	n ₁₂₀	n ₂₄₀	n ₃₀₀	n _v	n _T	$ \bar{Q} $	P $\bar{Q}$	T
Crane	101,362	2864	4	47	147	135	50	379	15,199	105,302	2652	600
			8	21	77	65	24	187	3951	109,493	2648	410
			12	5	57	49	6	117	1828	113,983	2520	370
			16	2	49	35	6	92	1065	118,057	2512	330
			20	4	40	34	4	82	704	121,936	2560	320
Letter_T	117,966	2100	24	3	32	26	3	64	506	126,204	2544	310
			4	168	135	129	168	600	17,480	121,105	2800	570
			8	20	68	64	19	171	4502	124,763	2288	410
			12	45	47	43	44	179	2057	128,262	2676	360
			16	10	37	31	10	88	1227	136,014	2320	330
Dancer	46,280	2997	20	0	32	24	1	57	805	139,430	2180	320
			24	1	26	20	1	48	551	137,428	2184	310
			4	75	161	141	82	459	7295	50,541	2772	530
			8	28	84	74	30	216	1967	54,511	2608	390
			12	11	56	52	10	129	951	59,298	2472	350
			16	16	46	38	17	117	565	62,631	2592	340
			20	4	33	27	4	68	387	67,030	2280	330
			24	8	29	23	8	68	274	68,340	2352	320

Table 1 continued

Object A	$ A $	n	g	n_{60}	n_{120}	n_{240}	n_{300}	n_v	n_T	$ \overline{Q} $	$p_{\overline{Q}}$	T
Bear	36, 450	1238	4	27	67	53	31	178	5511	38, 181	1208	130
			8	7	39	25	11	82	1443	39, 990	1144	90
			12	4	24	12	7	47	690	43, 024	1116	90
			16	7	15	13	5	40	395	43, 786	1136	80
			20	1	20	10	3	34	273	47, 285	1100	70
			24	3	18	8	5	34	189	47, 139	1176	70

$|A|$ = number of object pixels; n = number of pixels on the boundary of A ; g = grid size; n_{θ} = number of vertices with internal angle θ ($= 60, 120, 240, 300$); n_v = total number of vertices of $\overline{Q}$; n_T = number of UGTs in outer cover; $|\overline{Q}|$ = number of pixels in $\overline{Q}$; T = CPU time in milliseconds; $p_{\overline{Q}}$ = perimeter of $\overline{Q}$

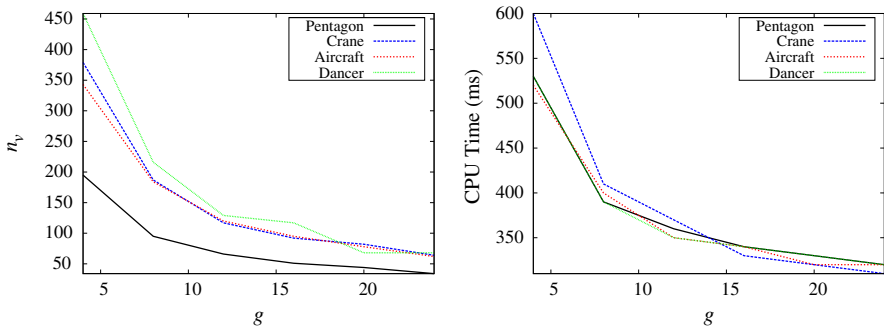


Fig. 8 Plots on (left) number of vertices and (right) CPU time versus g for aircraft, dancer, Crane, and pentagon images

roughly along the grid lines of the triangular grid, which results in fewer vertices of the triangular cover compared to the isothetic cover; and here also, the area measure of the triangular cover is less than that of the isothetic cover. Thus, the triangular covers are particularly effective, both in terms of vertex count and area measure, when the object boundary has some alignment with the underlying grid. An isothetic cover, in general, may have a smaller set of vertices, and so its computational time may be less. On the other hand, a triangular cover usually requires less area to encompass the object; and hence, it offers more compactness, although it can have more vertices and may require more computational time (Figs. 10, 11, 12).

6 Shape retrieval

An approximate shape of an object is captured by the triangular cover from which a chain code representing the shape of the object is derived which can be used for shape analysis. For this purpose we have put to use the Levenshtein distance [21] which is the minimum number of insertions, deletions or substitutions required to convert one string i.e., chain code into other [32].

- A database, $\mathcal{D}$, consisting of 125 shapes, some downloaded from the Internet and some selected from MPEG7 database, is created.
- All the shapes of $\mathcal{D}$ are normalized to the size 256×256 .
- The chain code, c_i , of each shape in $\mathcal{D}$, is generated using Algo. 3.4.
- For the query image, the chain code, called the query code, q , is generated.
- The Levenshtein distances, l_i 's, between the query code, q , and the chain code of each shape, c_i , are computed.
- The l_i 's, the dissimilarity measures, are sorted in ascending order. Smaller value of Levenshtein distance indicates the shape is more similar.
- The first k shapes from the sorted list are reported which are in decreasing order of similarity.

Experimental results have been shown in Table 2 where shapes similar to the query image have been retrieved from a given shape database in the increasing order of their Levenshtein distances between their chaincodes.

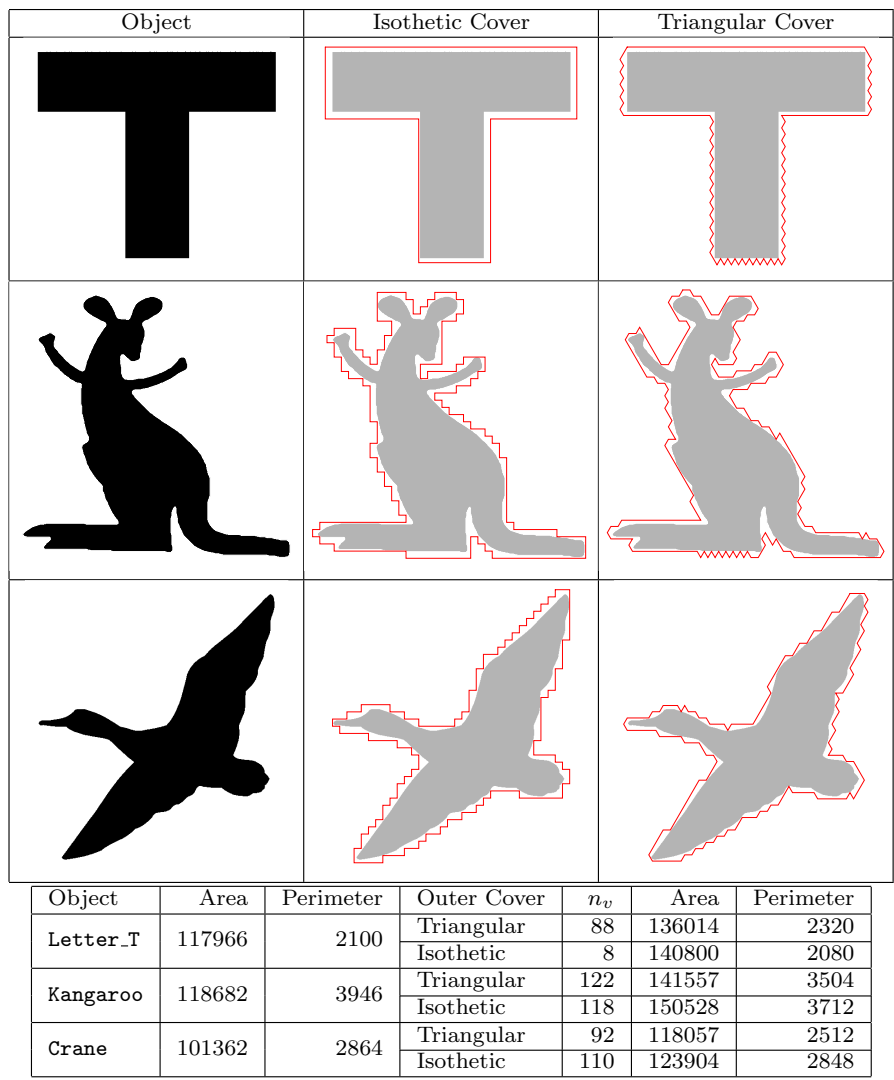


Fig. 9 Comparison of the triangular covers produced by the proposed algorithm with the isothetic covers produced by the algorithm in [3], all for $g = 16$. (n_v = number of vertices.)

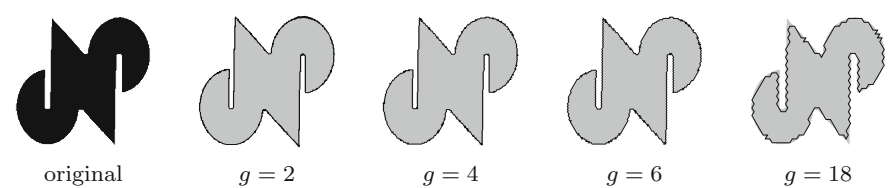


Fig. 10 The ITCs of a Logo image corresponding to different grid sizes

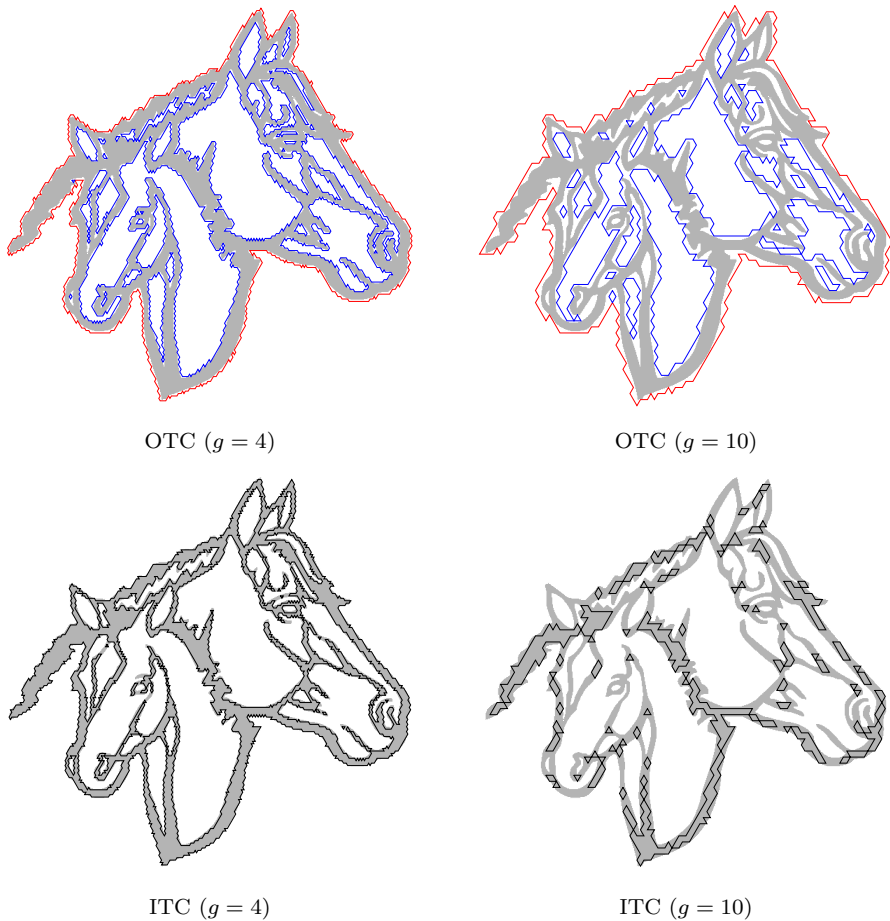


Fig. 11 The OTCs and ITCs corresponding to the image Horse for $g = 4$ and $g = 10$

Another set of retrieval results is presented in Table 3 where iso-taxicab distance between successive vertices of the outer triangular cover has been used. The outer triangular cover of a digital object is constructed and the iso-taxicab distances over all pairs of consecutive vertices (starting from the top-left vertex) of the cover constitute a code, $b = l_1 l_2 \dots l_k$, which we refer to as *distance-code*. The distance-code is used as the shape descriptor of the digital object. Thus, the distance-codes of the objects in the database are determined. Then the Levenstein distance between distance-code of the query object, b_q and those of the objects in the database, b_j s, are calculated to find the similarity. The method for calculating the Levinstein Distance is modified in the following way: two distances belonging to the distance-codes of the query and database object (say, l_q of the query and l_j of the database image) are considered to be same if $(1 - \alpha) \times l_j < l_q < (1 + \alpha) \times l_j$. The result of one such query is shown in Table 3 for $\alpha = 0.10$.

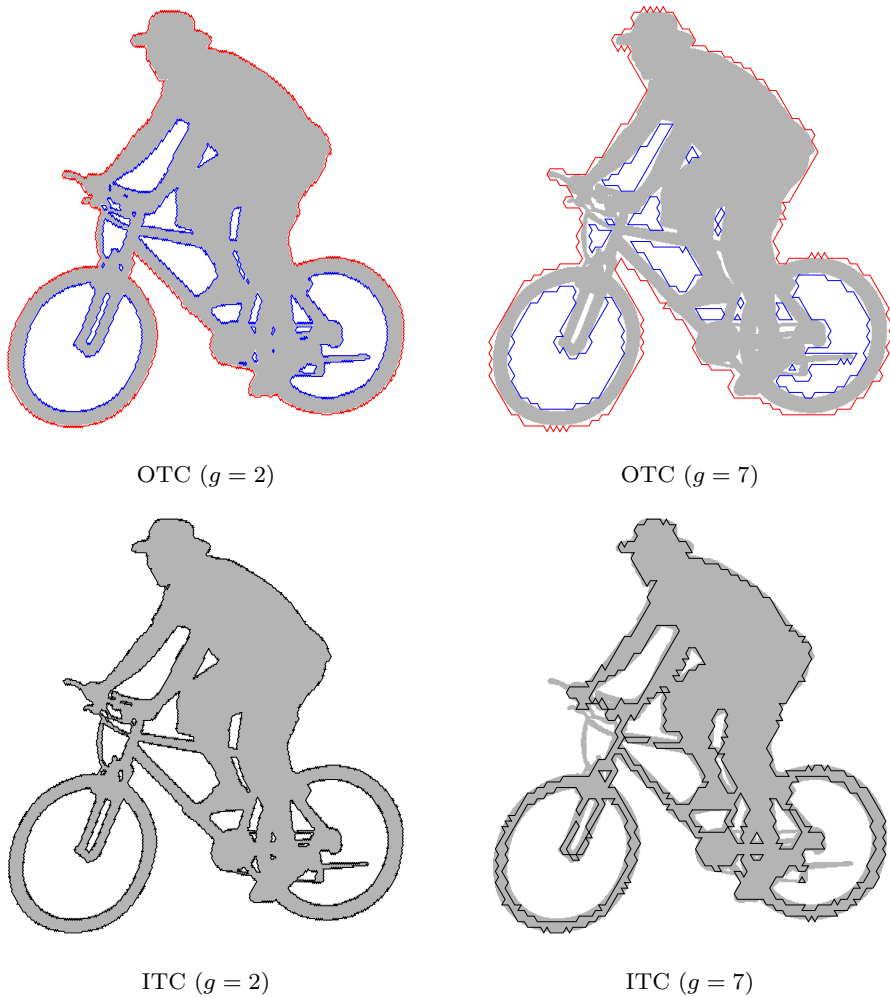


Fig. 12 The OTCs and ITCs corresponding to the image *Cyclist* for $g = 2$ and $g = 7$

7 Conclusion

We have proposed an algorithm for constructing a minimum-area cover of a digital object on a triangular grid. It can be used to determine all outer triangular polygons including the outer triangular hole polygons in a digital object with multiple components and holes. The minimum-area cover provides a compact geometric information of the object, which is useful in shape characterization along with those obtained from orthogonal or isothetic covers. Further, its runtime can be controlled by tuning the grid size, which offers a mechanism of trading off speed with granularity of the object boundary. Experimental results demonstrate that a triangular cover can serve as an approximate estimate of shape complexity and of similar other signatures used for characterizing a digital object.

Table 2 Leftmost column in each row shows the query image and remaining columns show the retrieved images in the increasing order of Levenshtein distance based on chain codes (written inside parenthesis)

	 (0)	 (18)	 (23)	 (23)	 (25)	 (27)
	 (0)	 (8)	 (9)	 (14)	 (15)	 (20)
	 (0)	 (6)	 (7)	 (8)	 (10)	 (24)
	 (0)	 (62)	 (68)	 (68)	 (68)	 (69)

Table 3 Leftmost is the query image and the subsequent images are the retrieved images in the increasing order of Levenshtein distance based on iso-taxicab distance (written inside parenthesis)

					
	(0)	(4)	(15)	(33)	(51)

There are several applications of triangular covers, one of them have been implemented and analysed as shown in Sect. 6 with results shown. Further analysis of this shape retrieval technique will be exploited in the near future. Triangular polygons can further be studied for digital convexity, medical image processing, developing triangular convex hull etc.

References

1. Beeson, M.: Triangle tiling I: the tile is similar to ABC or has a right angle (2012). arXiv preprint [arXiv:1206.2231](https://arxiv.org/abs/1206.2231)
2. Birch, C.P.D., Oom, S.P., Beecham, J.A.: Rectangular and hexagonal grids used for observation, experiment and simulation in ecology. *Ecol. Model.* **206**(3), 347–359 (2007)
3. Biswas, A., Bhowmick, P., Bhattacharya, B.B.: Construction of isothetic covers of a digital object: a combinatorial approach. *J. Vis. Commun. Image Represent.* **21**(4), 295–310 (2010)
4. Bodini, O., Rémila, E.: Tilings with trichromatic colored-edges triangles. *Theor. Comput. Sci.* **319**(1), 59–70 (2004)
5. Butler, S., Chung, F., Graham, R., Laczkovich, M.: Tiling polygons with lattice triangles. *Discrete Comput. Geom.* **44**(4), 896–903 (2010)
6. Clason, R.G.: Tiling with golden triangles and the penrose rhombs using logo. *J. Comput. Math. Sci. Teach.* **9**(2), 41–53 (1989)
7. Conway, J.H., Lagarias, J.C.: Tiling with polyominoes and combinatorial group theory. *J. Comb. Theory Ser. A* **53**(2), 183–208 (1990)
8. Daniel, H., Tom, K., Elmar, L.: Exploring simple triangular and hexagonal grid polygons online (2010). arXiv preprint [arXiv:1012.5253](https://arxiv.org/abs/1012.5253)

9. Das, B., Dutt, M., Biswas, A., Bhowmick, P., Bhattacharya, B.B.: A combinatorial technique for construction of triangular covers of digital objects. In: Proceedings of 16th International Workshop on Combinatorial Image Analysis (IWCIA 14), LNCS 8466, pp. 76–90. Springer (2014)
10. Freeman, H.: Algorithm for generating a digital straight line on a triangular grid. *IEEE Trans. Comput.* **100**(2), 150–152 (1979)
11. Gardner, M.: *Knotted Doughnuts and Other Mathematical Entertainments*. Freeman and Company, New York (1986)
12. Goodman-Strauss, C.: Regular production systems and triangle tilings. *Theor. Comput. Sci.* **410**(16), 1534–1549 (2009)
13. Her, I.: Geometric transformations on the hexagonal grid. *IEEE Trans. Image Process.* **4**(9), 1213–1222 (1995)
14. Innchyn, H.: Geometric transformations on the hexagonal grid. *IEEE Trans. Image Process.* **4**(9), 1213–1222 (1995)
15. Klette, R., Rosenfeld, A.: *Digital Geometry: Geometric Methods for Picture Analysis*. Morgan Kaufmann, San Francisco (2004)
16. Kocayusufoğlu, İ.: Trigonometry on iso-taxicab geometry. *Math. Comput. Appl.* **5**(3), 201–212 (2000)
17. Kocayusufoğlu, İ., Ada, T.: On the iso-taxicab trigonometry. *Appl. Sci.* **8**, 101–111 (2006)
18. Kocayusufoğlu, İ., Özdamar, E.: Connections and minimizing geodesics of taxicab geometry. *Math. Comput. Appl.* **5**(3), 191–200 (2000)
19. Krause, E.F.: *Taxicab Geometry*. Addison-Wesley, Menlo Park, NJ (1975)
20. Laczkovich, M.: Tilings of convex polygons with congruent triangles. *Discrete Comput. Geom.* **48**(2), 330–372 (2012)
21. Levenshtein, A.: Binary codes capable of correcting deletions, insertions and reversals. *Sov. Phys. Dokl.* **10**(8), 707–710 (1966)
22. Nagy, B.: Neighbourhood sequences in different grids. Ph.D. thesis, University of Debrecen (2003)
23. Nagy, B.: Characterization of digital circles in triangular grid. *Pattern Recognit. Lett.* **25**(11), 1231–1242 (2004)
24. Nagy, B.: Generalised triangular grids in digital geometry. *Acta Mathematica Academiae Paedagogicae Nyíregyháziensis* **20**(1), 63–78 (2004)
25. Nagy, B.: Cellular topology on the triangular grid. In: Proceedings of 15th International Workshop on Combinatorial Image Analysis (IWCIA 12), LNCS 7655, pp. 143–153. Springer (2012)
26. Nagy, B., Barczy, K.: Isoperimetrically optimal polygons in the triangular grid with Jordan-type neighbourhood on the boundary. *Int. J. Comput. Math.* **90**(8), 1–24 (2012)
27. Rosenfeld, A., Kak, A.: *Digital Picture Processing*, vol. 1. Elsevier, Amsterdam (2014)
28. Shimizu, K.: Algorithm for generating a digital circle on a triangular grid. *Comput. Graph. Image Process.* **15**(4), 401–402 (1981)
29. Sowell, K.O.: Taxicab geometry—a new slant. *Math. Mag.* **62**, 238–248 (1989)
30. Subramanian, K.G., Wiederhold, P.: Generative models for pictures tiled by triangles. *Sci. Technol.* **15**(3), 246–265 (2012)
31. Sury, B.: Group theory and tiling problems. *Symmetry Multi Discip. Perspect.* **16**(16), 97–117 (2011)
32. Wagner, R.A., Fischer, M.J.: The string-to-string correction problem. *J. ACM (JACM)* **21**(1), 168–173 (1974)
33. Wüthrich, C.A., Stucki, P.: An algorithmic comparison between square-and hexagonal-based grids. *CVGIP Graph. Models Image Process.* **53**(4), 324–339 (1991)